Guia de implementação do banco de dados do WellFlix

Este guia organiza o trabalho da equipe para transformar as entidades definidas em Especificação de Bancos de dados.md em um banco PostgreSQL e conectá-lo ao projeto. A especificação existente é a referência para o escopo funcional; as orientações abaixo detalham como implementá-la de forma relacional e integrada.

1. Tecnologias e responsabilidades

Tecnologia	Uso
JavaScript com Node.js	Implementar o backend/API que recebe as requisições do frontend e aplica as regras do sistema.
Express	Definir rotas HTTP da API, como cadastro, consulta de vídeos e criação de trilhas.
PostgreSQL	Persistir usuários, perfis, vídeos, assinaturas, trilhas e suas relações.
SQL	Criar e alterar tabelas, restrições, índices e consultas.
React/Vite existente	Apresentar telas e chamar a API por HTTP; não deve se conectar diretamente ao banco.

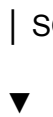
O backend pode acessar o PostgreSQL usando o pacote `pg` e consultas SQL parametrizadas. Mantenham as dependências do backend em `server/package.json`, separadas das dependências do frontend na raiz. A equipe deve escolher e compartilhar um único processo de migrações SQL antes de começar a criar tabelas; não mantenham cópias divergentes do schema nem façam a API criar tabelas automaticamente ao iniciar.

2. Arquitetura

Navegador (React/Vite)



API Node.js/Express



SQL parametrizado

PostgreSQL hospedado externamente

Arquivos de vídeo ──► armazenamento de arquivos/objetos

(o banco guarda apenas a chave/localização)

O navegador nunca recebe credenciais do PostgreSQL. As credenciais ficam em variáveis de ambiente do backend e não devem ser commitadas. O frontend chama a API; a API valida os dados e permissões, executa as consultas e responde em JSON.

3. Organização de pastas sugerida

Manter a aplicação existente em `src/` e adicionar um backend separado:

```
src/                    # frontend React/Vite existente
server/
  package.json          # dependências e comandos do backend
  .env.example          # nomes das variáveis, sem valores secretos
  src/
    app.js              # configuração do Express
    server.js           # inicialização HTTP
    routes/             # definição de endpoints
    controllers/        # leitura da requisição e resposta HTTP
    services/           # regras de negócio
    repositories/       # acesso SQL ao PostgreSQL
    middleware/         # autenticação, autorização e erros
    db/
      pool.js           # conexão compartilhada com pg
      migrations/       # SQL versionado, aplicado em ordem
```

```
seeds/                  # dados iniciais controlados
```

Fluxo esperado: `rota → controller → service → repository → PostgreSQL`. Controllers não devem conter SQL; repositories não devem conhecer detalhes de HTTP. Para escopos pequenos, a equipe pode simplificar a separação, mas deve manter essas responsabilidades claras.

4. Convenções comuns para todas as tabelas

Estas convenções foram acordadas por toda a equipe e devem ser seguidas por todos os integrantes em todas as tabelas e migrações. Uma alteração só deve divergir delas se a equipe revisar e aprovar a mudança em conjunto.

- Usar nomes de tabelas e colunas em português do Brasil, em `snake_case` e com nomes descritivos; por exemplo, `profissionais`, `categorias_profissional` e `data_criacao`.
- Usar `id UUID PRIMARY KEY` nas entidades principais. Não usar CPF, e-mail ou nome como chave primária.
- Referenciar registros com chaves estrangeiras (`FOREIGN KEY`) e usar o mesmo tipo de dado nas duas colunas relacionadas.
- Incluir `data_criacao TIMESTAMPTZ NOT NULL DEFAULT now()` nas entidades principais. Adicionar `data_atualizacao` quando o registro puder ser editado.
- Definir `NOT NULL`, `UNIQUE`, `CHECK` e `ON DELETE` explicitamente conforme as regras do dado; não depender apenas da validação da interface.
- Criar índices nas chaves estrangeiras e colunas usadas com frequência em filtros, como situação, categoria e data.
- Guardar datas com fuso (`TIMESTAMPTZ`) e durações como quantidade inteira de segundos.
- Evitar listas em uma única coluna, campos com JSON usado para representar relações, ou cópias de dados que possam ser obtidos por consulta.
- Para operações com várias gravações relacionadas, usar transações.

IDs devem ser gerados por uma estratégia uniforme: PostgreSQL (`gen_random_uuid()`) ou aplicação Node.js, definida pela equipe. Não misturar estratégias sem uma razão documentada. Os nomes de tabelas dos exemplos e das tarefas abaixo seguem a convenção obrigatória em português.

5. Divisão das implementações por integrante

Esta divisão segue os responsáveis listados em Especificação de Bancos de dados.md. Os nomes sugeridos devem permanecer consistentes em migrations, consultas SQL, repositories e documentação.

Integrante	Tabelas sugeridas	Escopo da implementação
Sophia — Atleta	`atletas`	Implementar os dados próprios do atleta e a chave estrangeira para a conta comum. Não incluir listas de assinaturas, vídeos, profissionais seguidos ou trilhas na linha do atleta.
Gabriel — Profissional e Moderador	`profissionais`, `categorias_profissional`, `seguimentos_profissional`; papel moderador em `papeis_usuario` e, se houver atributos próprios, `moderadores`	Implementar perfil profissional, categorias e seguidores. Integrar o papel moderador à estrutura compartilhada de `papeis_usuario`; criar `moderadores` apenas se houver dados exclusivos além dos mantidos na conta.
Luís — Vídeo	`videos`, `revisoes_video`, `categorias_video` (somente se um vídeo puder ter várias categorias)	Implementar metadados, profissional autor, categoria(s), duração, localização do arquivo e situação de publicação. Registrar decisões de moderação vinculadas ao moderador e ao vídeo.
Maria Eduarda — Assinatura e Trilhas	`planos`, `assinaturas`, `categorias_assinatura`, `trilhas`, `videos_trilha`, `progresso_trilha`	Implementar planos e assinaturas vinculados ao atleta, incluindo categorias selecionadas; implementar trilhas, ordem dos vídeos, privacidade e progresso. Os planos atuais em `src/data/plans.js` podem virar dados iniciais de `planos`.
Equipe — infraestrutura compartilhada	`usuarios`, `papeis_usuario`, `categorias`	A especificação não atribui essas entidades a uma pessoa. A equipe deve designar um mantenedor antes das migrations dependentes. Todos devem

Integrante	Tabelas sugeridas	Escopo da implementação
		usar os mesmos registros e IDs, sem criar cópias locais.
Equipe — comentários e avaliações	`comentarios`, `avaliacoes`	A especificação cita esses recursos, mas não indica um responsável. Definir em conjunto quem os implementará; cada registro deve referenciar usuário e vídeo. Avaliar se haverá uma avaliação por usuário por vídeo.

Como converter os campos do rascunho em relações

- `Profissional.Categorias`: tabela de ligação `categorias_profissional`, com uma linha por profissional e categoria.
- `Profissional.ID Vídeo`: não precisa de lista duplicada; cada vídeo aponta para o profissional em `videos.profissional_id`.
- `Profissional.Seguidores ID`: `seguimentos_profissional`, com `profissional_id` e `seguidor_id`.
- `Atleta.ID Assinatura`: cada linha de `assinaturas` aponta para o atleta. Se houver histórico, um atleta pode ter várias assinaturas ao longo do tempo.
- `Atleta.ID Vídeo`: não armazenar como lista no perfil. Consultar vídeos por avaliações, comentários, progresso ou outra relação de negócio.
- `Atleta.ID Trilha`: cada trilha aponta para seu dono em `atleta_id`; um atleta pode ter várias trilhas.
- `Vídeo.Categorias`: se cada vídeo tiver uma categoria, referenciar `categorias` diretamente. Se puder ter várias, criar `categorias_video`.
- `Vídeo.Comentários` e `Vídeo.Avaliações`: usar `comentarios` e `avaliacoes`, não arrays ou JSON na linha de vídeo.
- `Trilhas.ID Vídeos`: `videos_trilha` com uma coluna `ordem` para manter a sequência.

6. Exemplos de estrutura relacional

Exemplos ilustrativos que seguem as convenções acordadas. Ajustar campos e regras à especificação validada pela equipe antes de incluí-los nas migrações.

```
CREATE TABLE usuarios (

  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  nome_completo VARCHAR(120) NOT NULL,
```

```
    email VARCHAR(254) NOT NULL UNIQUE,
    cpf CHAR(11) UNIQUE,
    hash_senha TEXT NOT NULL,
    data_criacao TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE categorias (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    nome VARCHAR(100) NOT NULL UNIQUE
);
CREATE TABLE profissionais (
    usuario_id UUID PRIMARY KEY REFERENCES usuarios(id) ON DELETE CASCADE,
    especialidade VARCHAR(150),
    biografia TEXT
);
CREATE TABLE categorias_profissional (
    profissional_id UUID NOT NULL
        REFERENCES profissionais(usuario_id) ON DELETE CASCADE,
    categoria_id UUID NOT NULL
        REFERENCES categorias(id) ON DELETE RESTRICT,
    PRIMARY KEY (profissional_id, categoria_id)
);
```

Uma relação muitos-para-muitos de trilhas e vídeos segue o mesmo princípio:

```
CREATE TABLE videos_trilha (
    trilha_id UUID NOT NULL REFERENCES trilhas(id) ON DELETE CASCADE,
```

```
    video_id UUID NOT NULL REFERENCES videos(id) ON DELETE RESTRICT,

    ordem INTEGER NOT NULL CHECK (ordem > 0),

    PRIMARY KEY (trilha_id, video_id),

    UNIQUE (trilha_id, ordem)

);
```

7. Regras de segurança e dados sensíveis

- Guardar somente `hash_senha`, nunca senha em texto puro. Usar biblioteca reconhecida para hash de senha no backend.
- CPF e e-mail são dados pessoais: coletar somente quando necessários, restringir acesso e não incluí-los em logs.
- Não armazenar número completo do cartão, código de segurança (CVV) ou credenciais de pagamento. Usar provedor de pagamento; guardar apenas identificadores externos e estado da transação quando necessário.
- Vídeos e imagens devem ficar em armazenamento de arquivos/objetos. A tabela `videos` guarda a chave ou URL protegida, não o arquivo binário.
- Usar parâmetros nas consultas SQL. Nunca concatenar valores fornecidos pelo usuário dentro de SQL.
- Validar autenticação e autorização no backend. Não confiar em papel, `usuario_id` ou permissões enviados pelo navegador sem verificá-los na sessão/token.
- Restringir CORS às origens necessárias e usar HTTPS no ambiente publicado.

Exemplo de consulta parametrizada com `pg`:

```
const result = await pool.query(

  "SELECT id, titulo FROM videos WHERE situacao = $1",

  ["publicado"],

);
```

8. Contrato de implementação para cada integrante

Cada responsável por uma entidade deve entregar:

1. **Migração SQL** com criação/alteração da tabela, restrições, índices e comentários necessários.
2. **Relações documentadas**: tabelas referenciadas, comportamento ao excluir e campos obrigatórios.
3. **Repository** com consultas parametrizadas para as operações acordadas.

4. **Service** com validações e regras de negócio que não são apenas restrições do banco.
5. **Rotas e controller**, se fizerem parte do escopo combinado daquela entrega.
6. **Testes** da migração e das operações principais, incluindo erros previsíveis como duplicidade e referência inexistente.
7. **Exemplo de requisição e resposta JSON** para o frontend consumir.

Antes de abrir uma alteração para revisão, a pessoa deve confirmar que as migrações aplicam em um banco vazio, funcionam na ordem prevista e não quebram tabelas de outros integrantes. Uma migração já compartilhada/aplicada não deve ser editada retroativamente; criar uma nova migração corretiva.

9. Ordem recomendada das migrações

As chaves estrangeiras determinam a ordem. Uma sequência inicial possível:

1. `usuarios` e `papeis_usuario`
2. `categorias` e `planos`
3. `atletas`, `profissionais` e, se necessário, `moderadores`
4. `categorias_profissional`
5. `videos` e seus vínculos com profissionais/categorias
6. `revisoes_video`, `comentarios` e `avaliacoes`
7. `seguimentos_profissional`
8. `assinaturas` e `categorias_assinatura`
9. `trilhas`, `videos_trilha` e `progresso_trilha`

As tabelas `usuarios`, `papeis_usuario` e `categorias` são infraestrutura compartilhada e precisam ser criadas ou atribuídas a um mantenedor antes das migrations dependentes. Se atletas, profissionais ou moderadores não tiverem atributos próprios além do papel, a equipe pode representá-los apenas em `papeis_usuario`; não criar estruturas vazias por duplicidade. Essa decisão precisa ser acordada por todos, pois afeta as chaves estrangeiras.

10. Processo de trabalho em equipe

1. Definir uma pessoa responsável por revisar convenções, nomes, IDs, estados e estratégia de migração.
2. Fazer uma revisão rápida das dependências entre tabelas antes de dividir as tarefas.
3. Atribuir a cada integrante uma ou mais tabelas e registrar quem é responsável por cada migração.
4. Integrar as migrações em uma única sequência; resolver conflitos de nomes e dependências antes de escrever consultas da API.
5. Compartilhar um banco de desenvolvimento comum ou cada integrante aplicar as mesmas migrações no próprio banco local.
6. Criar seeds somente para dados necessários ao desenvolvimento, como categorias e planos; não misturar dados pessoais reais.

7. Revisar schema e endpoints em conjunto e validar o fluxo ponta a ponta com o frontend.

11. Integração gradual com o frontend existente

Atualmente, partes da interface são demonstrações com dados em `src/data/content.js`, `src/data/plans.js`, estado React ou `localStorage`. A migração para dados persistidos pode ser feita por funcionalidade:

1. Cadastro/login e identidade do usuário.
2. Catálogo de vídeos e perfis de profissionais.
3. Envio de vídeo e fila de moderação.
4. Comentários, avaliações, seguidores e trilhas.
5. Planos, assinaturas e integração do pagamento.

O backend deve expor endpoints JSON estáveis, por exemplo `GET /api/videos`, `POST /api/videos`, `POST /api/videos/:id/reviews` e `POST /api/subscriptions`. O frontend deve consumir esses endpoints; as páginas não devem abrir conexão PostgreSQL nem conter SQL.

12. Critérios de conclusão do banco

- Todas as tabelas e relações da especificação estão representadas sem listas relacionais em uma coluna.
- As migrações criam o schema em banco vazio e são aplicáveis na ordem documentada.
- Chaves primárias, estrangeiras, restrições e índices foram revisados.
- Contas e permissões são verificadas no backend; dados sensíveis não são expostos.
- Arquivos de mídia e dados de cartão não são armazenados indevidamente no PostgreSQL.
- Pelo menos um fluxo de ponta a ponta persiste dados na API e os recupera após recarregar a página.